

Software Engineering

Dr. Md. Nasim Adnan

- He had the classic look of a senior executive for a major software company—mid-40s, slightly graying at the temples, trim and athletic, with eyes that penetrated the listener as he spoke.
- But what he said shocked me. “Software is *dead*.”
- I blinked with surprise and then smiled. “You’re joking, right? The world is driven by software and your company has profited handsomely because of it. It isn’t dead! It’s alive and growing.”
- He shook his head emphatically. “No, it’s dead . . . at least as we once knew it.”
- I leaned forward. “Go on.”

- He spoke while tapping the table for emphasis. “The old-school view of software—you buy it, you own it, and it’s your job to manage it—that’s coming to an end.
- Today, with Web 2.0 and pervasive computing coming on strong, we’re going to be seeing a completely different generation of software. It’ll be delivered via the Internet and will look exactly like it’s residing on each user’s computing device . . . but it’ll reside on a far-away server.”
- Is software *really dead? If it was, you wouldn’t be reading this book!*

- Computer software continues to be the single most important technology on the world stage. And it's also a prime example of the law of unintended consequences.
- Fifty years ago no one could have predicted that software would become an indispensable technology for business, science, and engineering.
- When Dijkstra married Maria in 1957, he was required as a part of the marriage rites to state his profession. He stated that he was a Programmer, which was unacceptable to the authorities, there being no such profession then in The Netherlands.

- No one could predict that millions of computer programs would have to be corrected, adapted, and enhanced as time passed.
- The burden of performing these “maintenance” activities would absorb more people and more resources than all work applied to the creation of new software.
- As software’s importance has grown, the software community has continually attempted to develop technologies that will make it easier, faster, and less expensive to build and maintain high-quality computer programs.

The Nature of Software

- Today, software takes on a dual role. It is a product, and at the same time, the vehicle for delivering a product. As a product, it delivers the computing potential embodied by computer hardware or more broadly, by a network of computers that are accessible by local hardware.
- Whether it resides within a mobile phone or operates inside a mainframe computer, software is an information transformer—producing, managing, acquiring, modifying, displaying, or transmitting information that can be as simple as a single bit or as complex as a multimedia presentation derived from data acquired from dozens of independent sources.

- As the vehicle used to deliver the product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs (software tools and environments).
- Software delivers the most important product of our time—*information*. It transforms personal data (e.g., an individual's financial transactions) so that the data can be more useful in a local context; it manages business information to enhance competitiveness; it provides a gateway to worldwide information networks (e.g., the Internet), and provides the means for acquiring information in all of its forms.

- Today, a huge software industry has become a dominant factor in the economies of the industrialized world. Teams of software specialists, each focusing on one part of the technology required to deliver a complex application, have replaced the lone programmer of an earlier era.
- And yet, the questions that were asked of the lone programmer are the same questions that are asked when modern computer-based systems are built:

- Why does it take so long to get software finished?
- Why are development costs so high?
- Why can't we find all errors before we give the software to our customers?
- Why do we spend so much time and effort maintaining existing programs?
- Why do we continue to have difficulty in measuring progress as software is being developed and maintained?



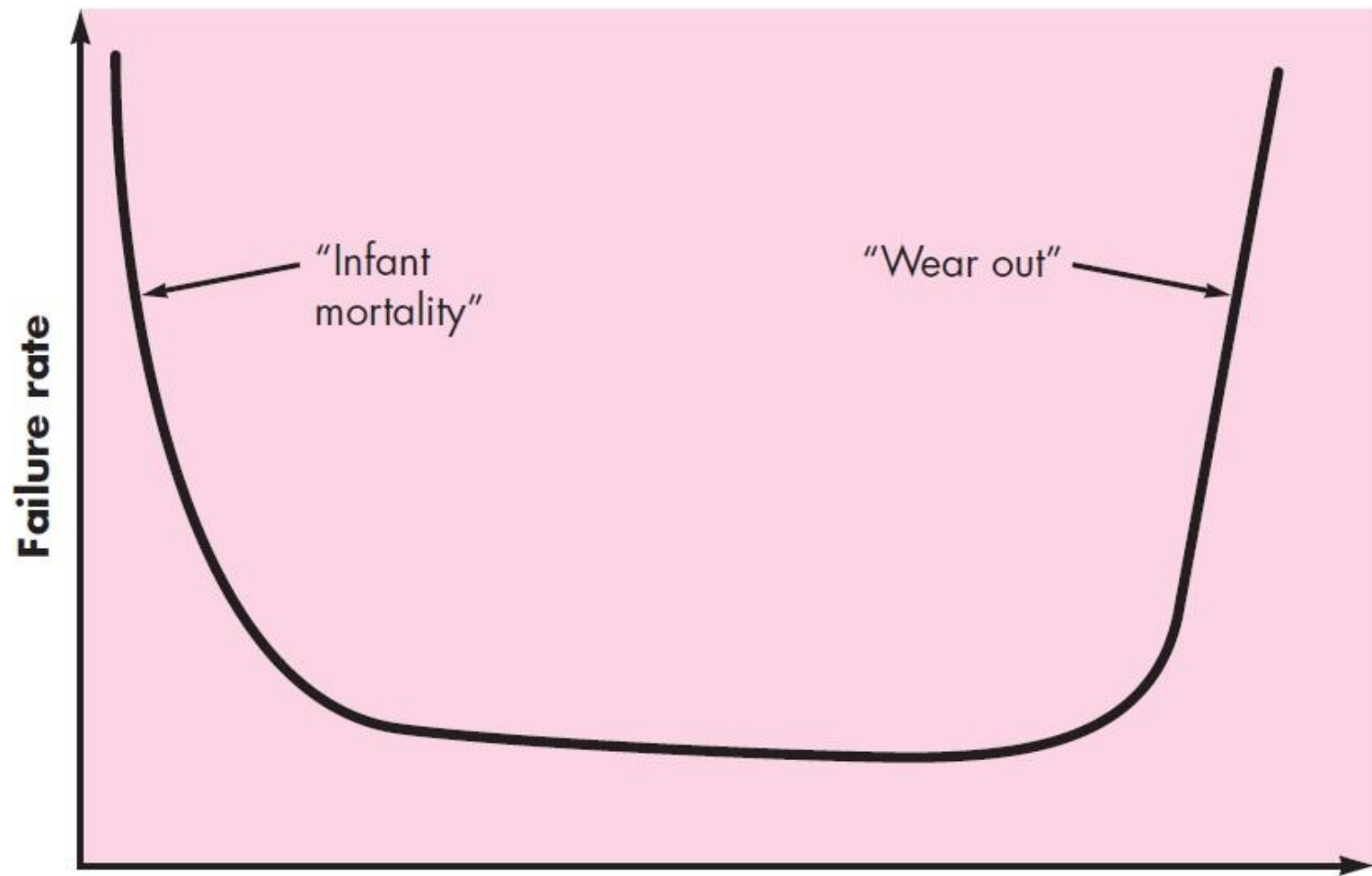
- Margaret Hamilton in 1969, standing next to listings of the software she and her MIT team produced for the Apollo project.

Defining Software

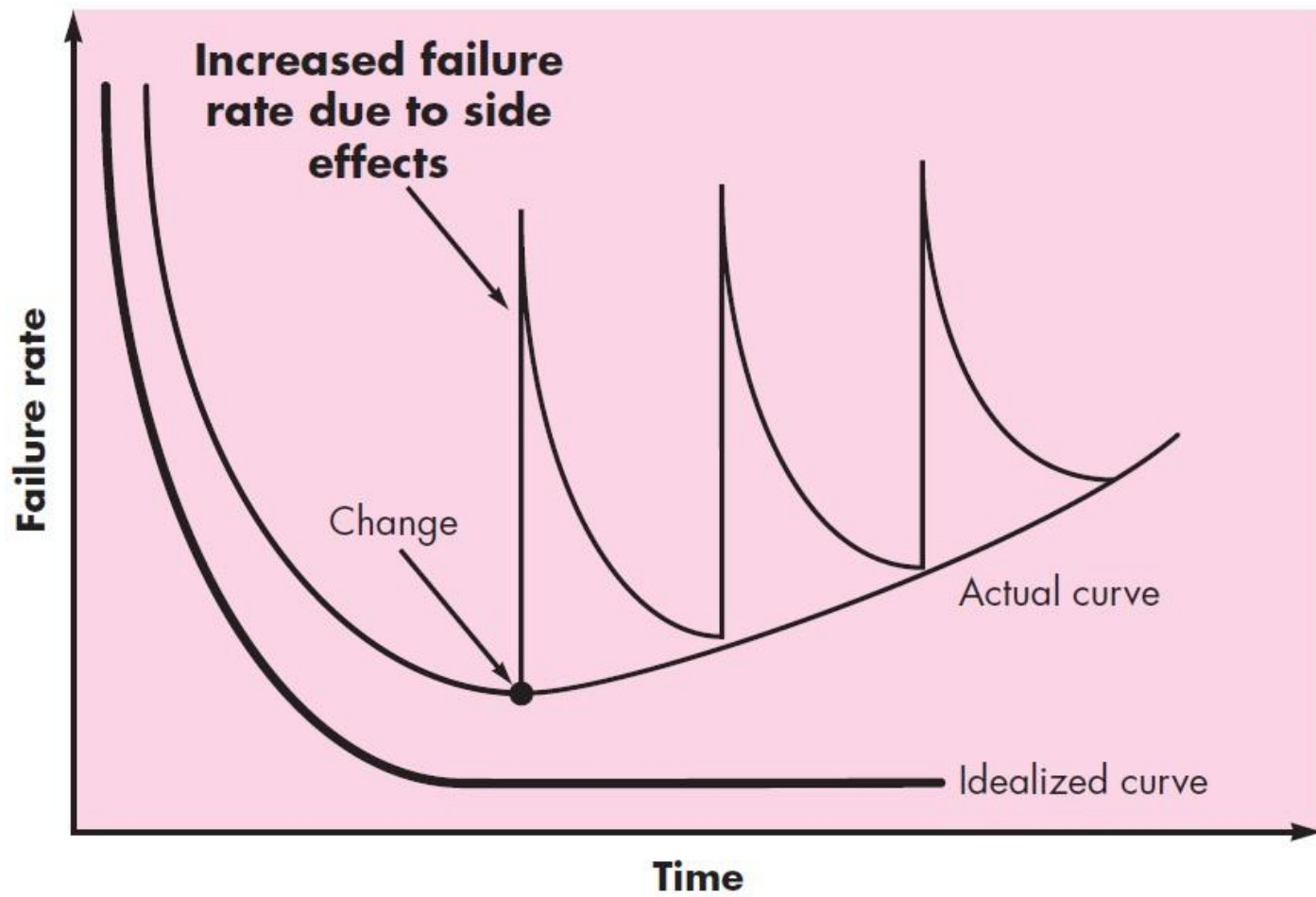
- Software is:
 - (1) instructions (computer programs) that when executed provide desired features, function, and performance;
 - (2) data structures that enable the programs to adequately manipulate information, and
 - (3) descriptive information in both hard copy and virtual forms that describes the operation and use of the programs.

- Software has characteristics that are considerably different than those of hardware:
 - *Software is developed or engineered; it is not manufactured in the classical sense*
 - *Software doesn't "wear out."*
 - *Although the industry is moving toward component-based construction, most software continues to be custom built.*

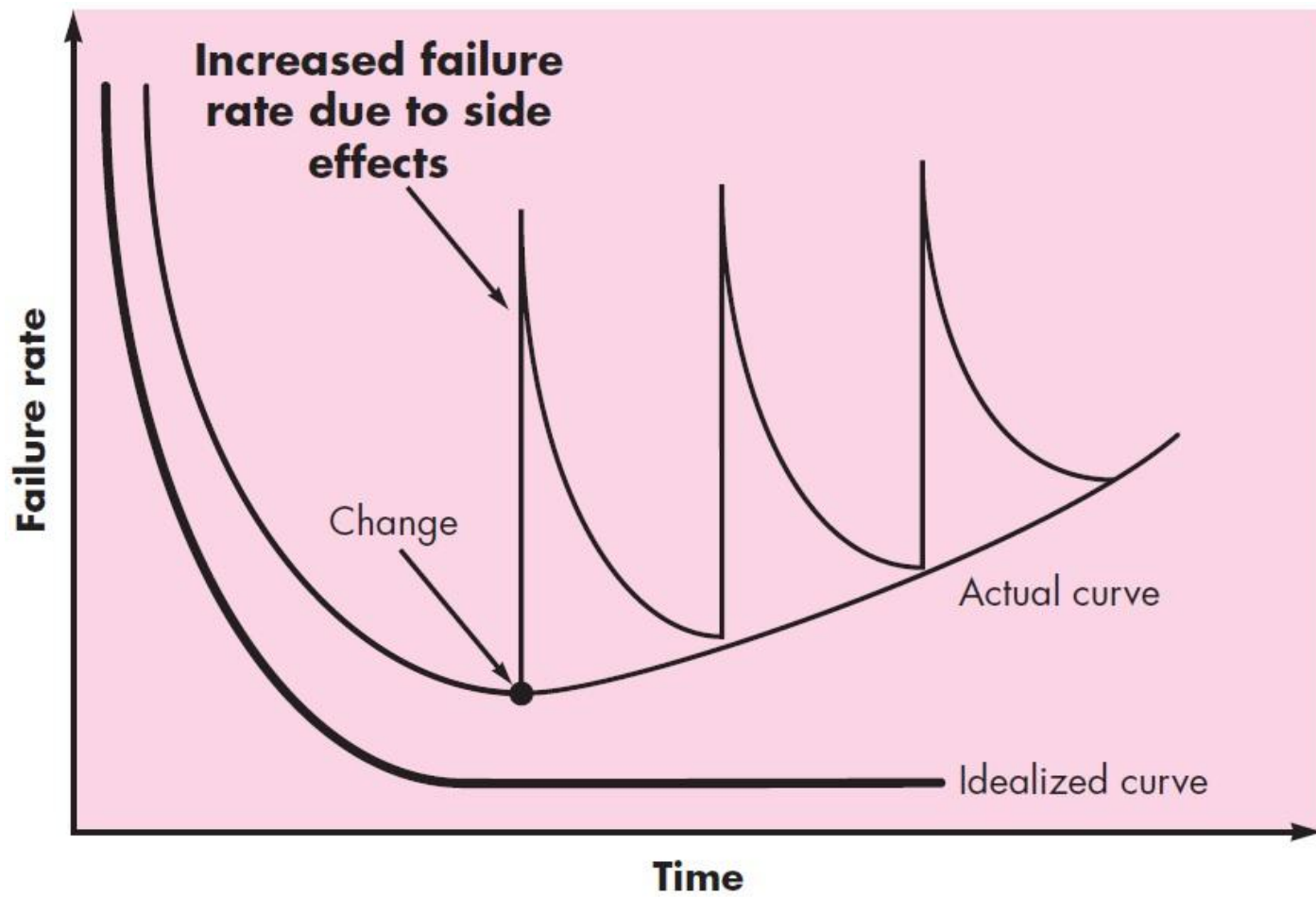
- The next Figure depicts failure rate as a function of time for hardware. The relationship, often called the “bathtub curve,” indicates that hardware exhibits relatively high failure rates early in its life (these failures are often attributable to design or manufacturing defects); defects are corrected and the failure rate drops to a steady-state level (hopefully, quite low) for some period of time.
- As time passes, however, the failure rate rises again as hardware components suffer from the cumulative effects of dust, vibration, abuse, temperature extremes, and many other environmental maladies. Stated simply, the hardware begins to *wear out*.



- Software is not susceptible to the environmental maladies that cause hardware to wear out. In theory, therefore, the failure rate curve for software should take the form of the “idealized curve” shown in the next Figure.
- Undiscovered defects will cause high failure rates early in the life of a program. However, these are corrected and the curve flattens as shown.
- The idealized curve is a gross oversimplification of actual failure models for software.
- However, the implication is clear—software doesn’t wear out. But it does *deteriorate!*



- As changes are made, it is likely that errors will be introduced, causing the failure rate curve to spike as shown in the “actual curve”.
- Before the curve can return to the original steady-state failure rate, another change is requested, causing the curve to spike again. Slowly, the minimum failure rate level begins to rise—the software is deteriorating due to change.



Software Application Domains

- System Software
- Application Software
- Engineering/Scientific Software
- Embedded Software
- Product-line Software
- Web Applications
- Artificial Intelligence Software

Software Engineering

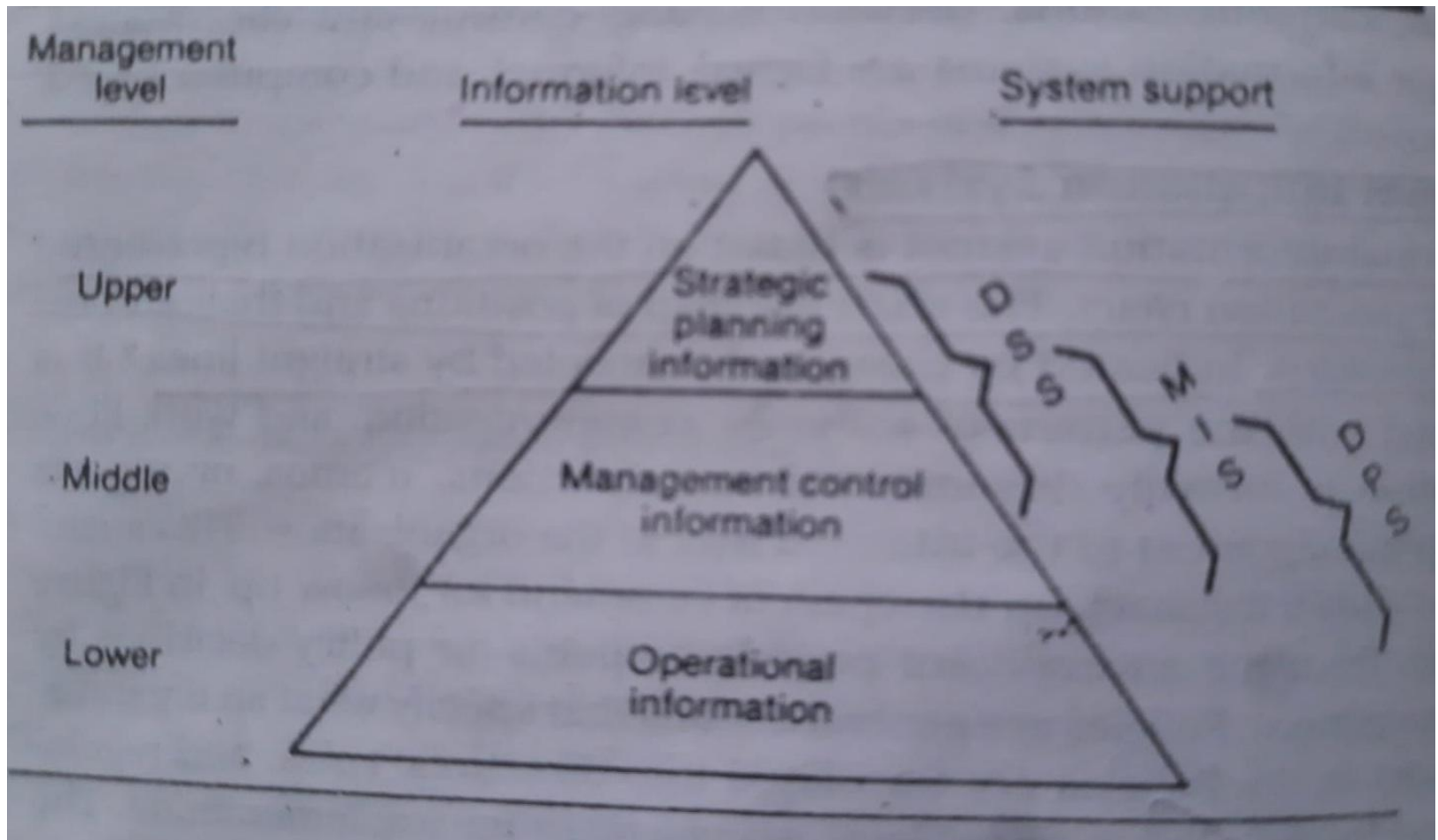
- In order to build software that is ready to meet the challenges of the twenty-first century, you must recognize a few simple realities:
 - *It follows that a concerted effort should be made to understand the problem before a software solution is developed.*
 - The information technology requirements demanded by individuals, businesses, and governments grow increasingly complex with each passing year (Income Tax)... Large teams of people now create computer programs that were once built by a single individual. *It follows that design becomes a pivotal activity.*

- *The hardest thing in the world to understand is the income tax.* – Albert Einstein

- Individuals, businesses, and governments increasingly rely on software for strategic and tactical decision making as well as day-to-day operations and control. If the software fails, people and major enterprises can experience anything from minor inconvenience to catastrophic failures. *It follows that software should exhibit high quality.* (Stock is Okay)
- As the perceived value of a specific application grows, the likelihood is that its user base and longevity will also grow. As its user base and time-in-use increase, demands for adaptation and enhancement will also grow. *It follows that software should be maintainable.*

Footnote

- DSS
 - Relates to long-range planning
 - Human resources changes
- MIS
 - Known to us all
 - Monthly/Yearly Sales analysis
- DPS
 - Enforce day to day rules and regulations of a business
 - Employee absence sheet, current stock of products



Focus Area: MIS

- A key element of MIS is the database. MIS cannot be implemented effectively without the help of a database
- A database is a nonredundant collection of interrelated data items that can be accessed through application programs and available to many users

Decision Making

❖ Elements?

Decision Making

- MIS – consolidates day to day transaction
 - Production MIS
 - Personnel MIS
 - Measures are consolidated according to Dimensions
- DSS – in my own view: results from all the available MIS systems, available external data ...

- All these realities lead to one conclusion: *software in all of its forms and across all of its application domains should be engineered. And that leads us to the topic of this book—software engineering.*
- Although hundreds of authors have developed personal definitions of software engineering, a definition proposed by Fritz Bauer still serves as a basis for discussion:
- [Software engineering is] the establishment and use of sound engineering principles in order to obtain economically software that is reliable and works efficiently on real machines.

The Software Process

- *A process is a collection of activities, actions, and tasks that are performed when some work product is to be created.*
 - *An activity strives to achieve a broad objective (e.g., communication with stakeholders) and is applied regardless of the application domain, size of the project, complexity of the effort, or degree of rigor with which software engineering is to be applied.*
 - *An action encompasses a set of tasks that produce a major work product (e.g., an architectural design model).*
 - *A task focuses on a small, but well-defined objective (e.g., conducting a unit test) that produces a tangible outcome.*

- In the context of software engineering, a process is *not a rigid prescription for how* to build computer software.
- Rather, it is an adaptable approach that enables the people doing the work (the software team) to pick and choose the appropriate set of activities, actions, and tasks.
- The intent is always to deliver software in a timely manner and with sufficient quality to satisfy those who have sponsored its creation and those who will use it.

Process Framework

- A generic process framework for software engineering encompasses five activities:
- **Communication.** Before any technical work can commence, it is critically important to communicate and collaborate with the customer (and other stakeholders).
- The intent is to understand stakeholders' objectives for the project and to gather requirements that help define software features and functions.

- **Planning.** Any complicated journey can be simplified if a map exists. A software project is a complicated journey, and the planning activity creates a “map” that helps guide the team as it makes the journey.
- The map—called a *software project plan*—defines the software engineering work by describing the technical tasks to be conducted, the risks that are likely, the resources that will be required, the work products to be produced, and a work schedule. (Padma Bridge)

- **Modeling.** Whether you're a landscaper, a bridge builder, an aeronautical engineer, a carpenter, or an architect, you work with models every day. You create a "sketch" of the thing so that you'll understand the big picture—what it will look like architecturally, how the constituent parts fit together, and many other characteristics.
- If required, you refine the sketch into greater and greater detail in an effort to better understand the problem and how you're going to solve it.
- A software engineer does the same thing by creating models to better understand software requirements and the design that will achieve those requirements.

- **Construction.** This activity combines code generation (either manual or automated) and the testing that is required to uncover errors in the code.
- **Deployment.** The software (as a complete entity or as a partially completed increment) is delivered to the customer who evaluates the delivered product and provides feedback based on the evaluation.

- As discussed previously, a generic process framework for software engineering defines five framework activities—**communication, planning, modeling, construction, and deployment.**
- We now describe the principles that focus on software engineering practice as a whole.

The First Principle: The Reason It All Exists

- A software system exists for one reason: to provide value to its users.
- All decisions should be made with this in mind. Before specifying a system requirement, before noting a piece of system functionality, before determining the hardware platforms or development processes, ask yourself questions such as: “Does this add real value to the system?”
- If the answer is “no,” don’t do it. All other principles support this one.

The Second Principle: KISS (Keep It Simple, Stupid!)

- Software design is not a haphazard process. There are many factors to consider in any design effort.
- All design should be as simple as possible.
- This facilitates having a more easily understood and easily maintained system.

The Third Principle: Maintain the Vision

- A clear vision is essential to the success of a software project.
- Without one, a project almost unfailingly ends up being “of two [or more] minds” about itself.
- Without conceptual integrity, a system threatens to become a patchwork of incompatible designs. (Backward Compatibility)

The Fourth Principle: What You Produce, Others Will Consume

- Seldom is an industrial-strength software system constructed and used in a vacuum.
- In some way or other, someone else will use, maintain, document, or otherwise depend on being able to understand your system.
- So, always specify, design, and implement knowing someone else will have to understand what you are doing.

- The audience for any product of software development is potentially large.
- Specify with an eye to the users. Design, keeping the implementers in mind. Code with concern for those that must maintain and extend the system. Someone may have to debug the code you write, and that makes them a user of your code.
- Making their job easier adds value to the system.

The Fifth Principle: Be Open to the Future

- A system with a long lifetime has more value. In today's computing environments, where specifications change on a moment's notice and hardware platforms are obsolete just a few months old, software lifetimes are typically measured in months instead of years.
- However, true “industrial-strength” software systems must endure far longer.
- To do this successfully, these systems must be ready to adapt to changes. Systems that do this successfully are those that have been designed this way from the start.

The Sixth Principle: Plan Ahead for Reuse

- Reuse saves time and effort. Achieving a high level of reuse is arguably the hardest goal to accomplish in developing a software system.
- The reuse of code and designs has been proclaimed as a major benefit of using object-oriented technologies.
- However, the return on this investment is not automatic. To leverage the reuse possibilities that object-oriented [or conventional] programming requires forethought and planning.

The Seventh principle: Think!

- This last principle is probably the most overlooked. Placing clear, complete thought before action almost always produces better results.
- When you think about something, you are more likely to do it right.

